> The front-end accepts the request because the WAF only looks for clear, rule-matching patterns like `Transfer-Encoding: chunked`. By adding another Transfer-Encoding header, the request becomes ambiguous, the rule does not trigger, and the WAF allows it even though the proxy later treats it as chunked.

🔒 If you want, next I can:

- Draw **exact decision flow diagram**
- Show **real WAF pseudo-rules**
- Explain **how defenders should fix this**

කියන්න 👍

---

yes, i asked this, i got my answer form it, thnks, give short note for it

මෙන්න **කෙටි note** එක (revision / exam / lab reference සඳහා) 👇

📌 Why Front-end (WAF/CDN) Accepts the Request in HTTP Request Smuggling

- **WAF/CDN "block or allow"** කරනේ **HTTP protocol parse** කරලා නෙවෙයි,
  ඒක **rule-based text inspection (regex / string matching)** මත.
- බොහෝ WAF rules **clear patterns** පමණක් block කරනවා
  (උදා: `Transfer-Encoding: chunked`).
- Attacker **multiple / invalid Transfer-Encoding headers**
  (`chunked` + `cow`) යොදන්නේ request එක **ambiguous** කරලා,
  WAF rule match වීම වැළැක්වීමට.
- Ambiguous request එකක් දැකලා බොහෝ WAFs:
  - rule trigger නොකරයි
  - **ALLOW** කර forward කරනවා.
- Request allow වුණාට පස්සේ,
  proxy / front-end parser එක real HTTP parsing කරලා
  `Transfer-Encoding: chunked` enable කරනවා.

- Back-end server එක එම header combination එක **invalid** කියලා
  `Transfer-Encoding` ignore කර `Content-Length` use කරනවා.

👉 **Different components parse the same request differently → request smuggling succeeds.**

🔑 One-line takeaway

> The front-end accepts the request because the WAF only blocks obvious `Transfer-Encoding: chunked` patterns, and ambiguity causes the rule check to fail while the proxy still enables chunked parsing.